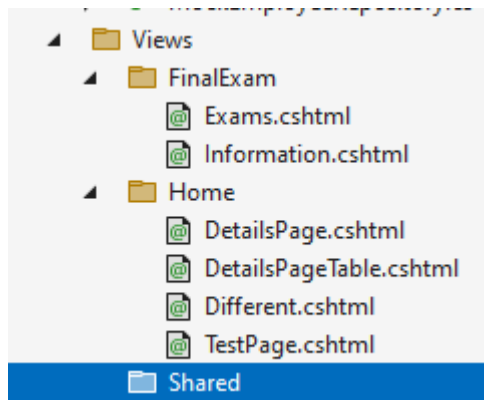


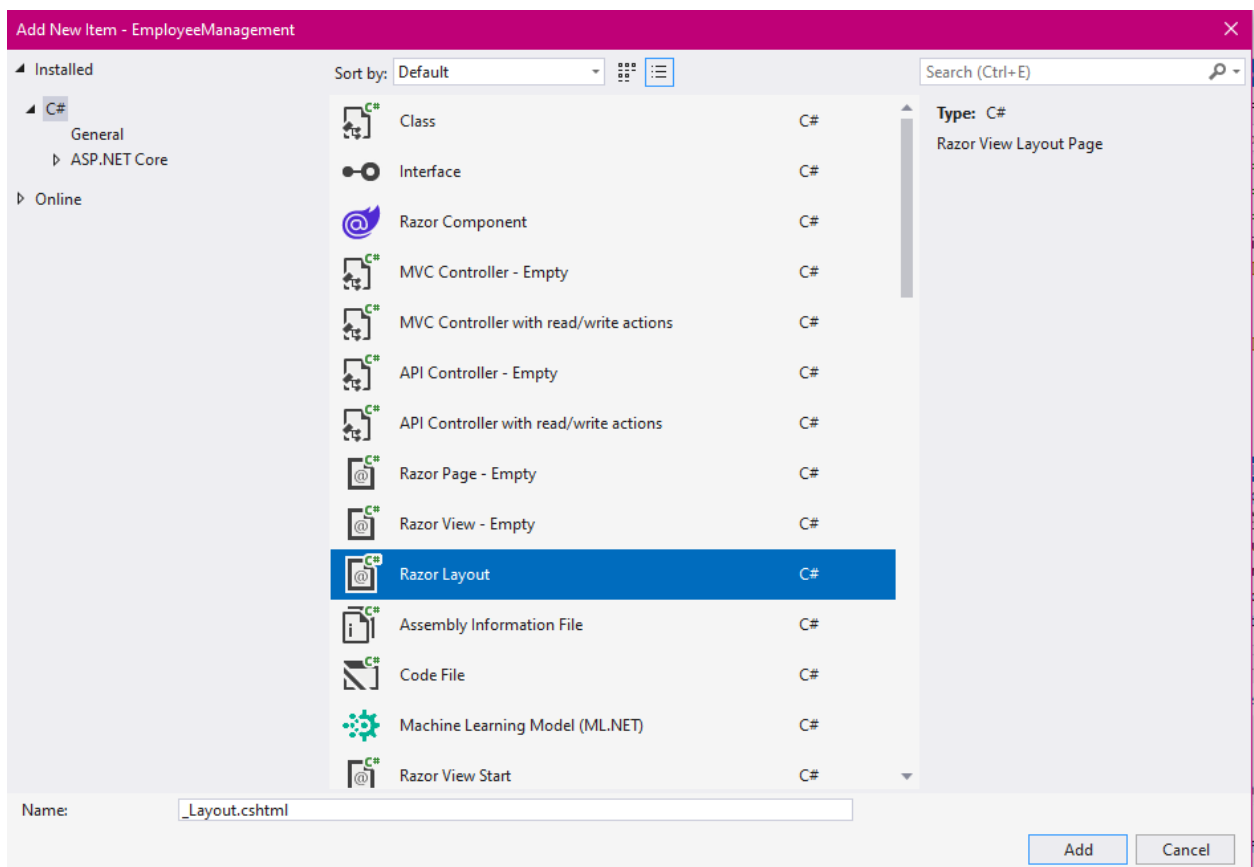
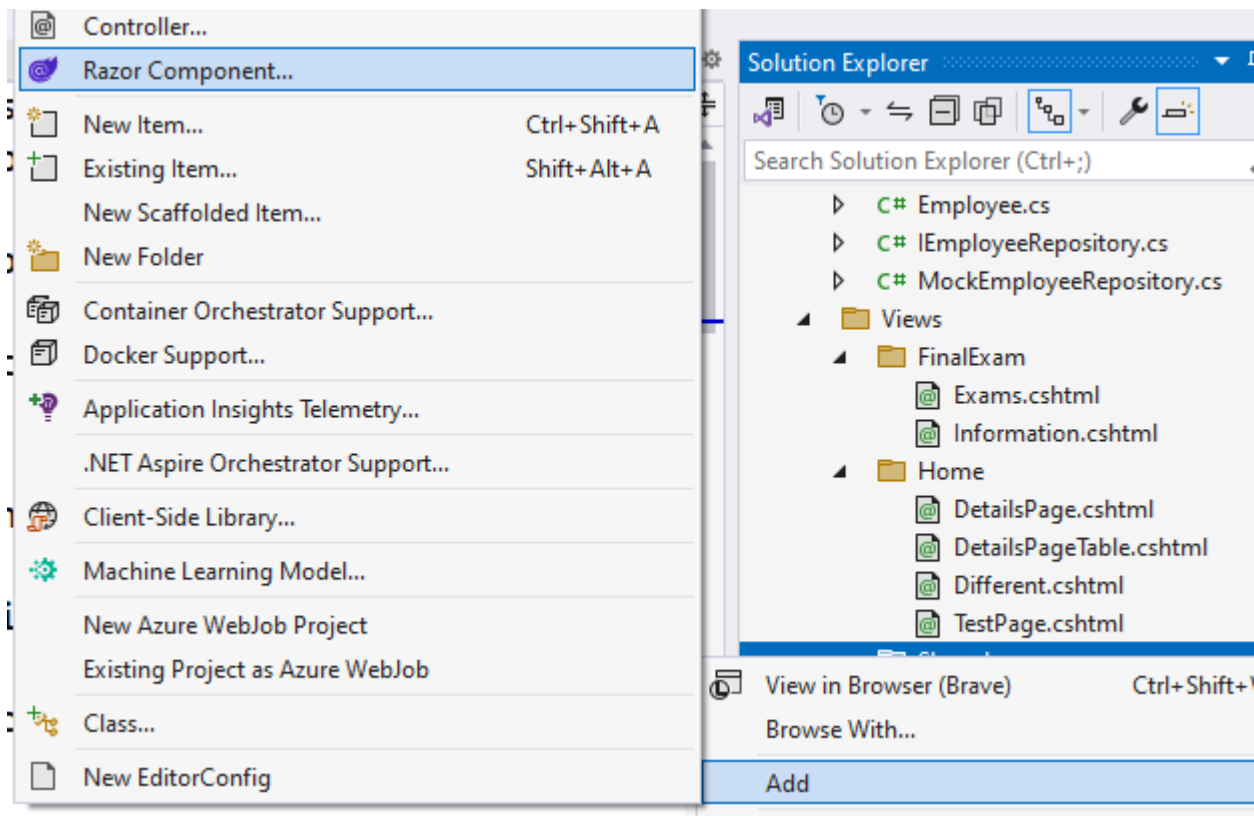
Layout

Layout View in ASP.NET Core MVC



Целта на тази Layout-структура е да не се налага повторение на HTML-кода за всяка страница в приложението, с което управлението би станало кошмар. В тях се повтаря голяма част от html-кода, който може да бъде изнесен в отделен Layout (както във Photoshop – всеки нов елемент в отделен Layout) и в .cshtml да остане само уникалната част, специфична само за конкретната страница. Така, ако се наложи да добавим или премахнем елемент от менюто например, няма да се налага да го правим във всички страници, а само на едно място в Layout, като промяната ще се отрази незабавно върху всички страници в цялото приложение. Обикновено поставяме всички Layout във Views/Shared-папка, така че нека я добавим. Всички Layout-оформления са със .cshtml разширения и може да ги възприемаме като главна рамка, master-page на останалите html-forms.





_Layout.cshtml по подразбиране започва с ' _ ', което показва, че целта на файла е да бъде обработен, без да бъде показван директно в браузъра (в противен случай, само разширението .cshtml може да ни подведе), по същият начин са следващите _ViewStart.cshtml, _ViewImports.cshtml.

_Layout.cshtml не е специфичен за даден контролер като предишните 2 - Index, Details, които принадлежат на Home, затова сме го поставили в папка Shared.

Възможно е да имаме няколко _Layout файла с различни имена в едно приложение, например един изглед за админа, а друг за всички останали потребители.

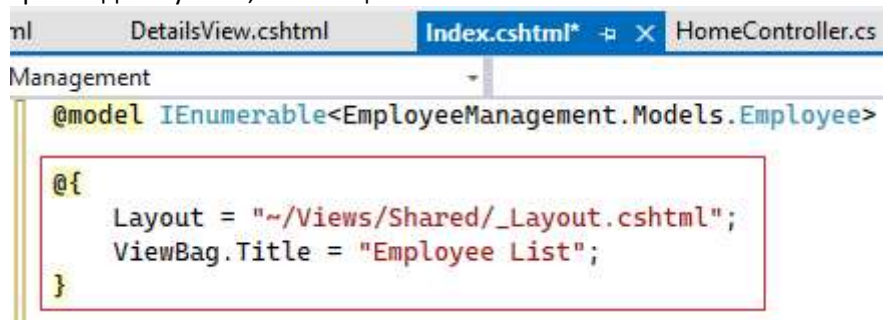


```
1 <!DOCTYPE html>
2
3 <html>
4 <head>
5     <meta name="viewport" content="width=device-width" />
6     <title>@ViewBag.Title</title>
7 </head>
8 <body>
9     <div>
10         @RenderBody()
11     </div>
12 </body>
13 </html>
14
```

Целият html-код, който е вложен по подразбиране като тагове html, head, body и се повтарят в останалите Index.cshtml, Details.cshtml, може да бъде премахнат, за да не се повтаря в тях. @ViewBag.Title – използва се за PageTitle, вместо да hard-code-нем заглавията на всички html-страници, по-добре да използваме ViewBag за толкова минимално количество информация, т.к. рисковете при грешки в него биха били с леки последици (просто няма да се зареди заглавието, а не да крашне цялата страница).

@RenderBody() – на негово място се инжектира целия специфичен индивидуален html-код, който евентуално бихме искали да се изведе в структурата на извиканите отделни страници.

За да се използва нашият файл с готова повтаряща се html-структура, в началото на всяка страница трябва да се укаже, че той ще се използва:



```
1 @model IEnumerable<EmployeeManagement.Models.Employee>
2
3 @{
4     Layout = "~/Views/Shared/_Layout.cshtml";
5     ViewBag.Title = "Employee List";
6 }
```

```
@{  
    Layout = "~/Views/Shared/_Layout.cshtml";  
    ViewBag.Title = "Employee List";  
}
```

Така може във всяка страница само да остане различното и характерно за нея без еднаквите тагове за <html><head><body>... Те остават само в _Layout.cshtml.

Не е най-удачното решение навсякъде да указваме, че се използва този _Layout, т.к. отново се получава много повтарящ се код, а за голям брой страници, това е трудно за редакция.

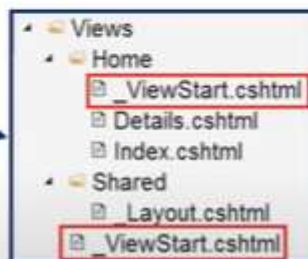
Layout View in ASP.NET Core MVC

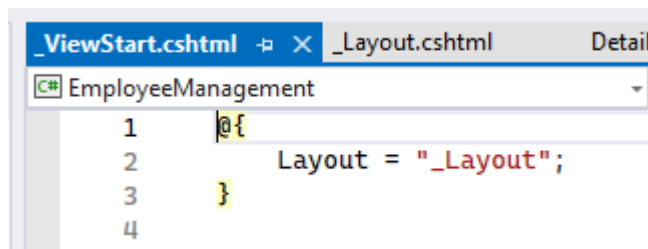
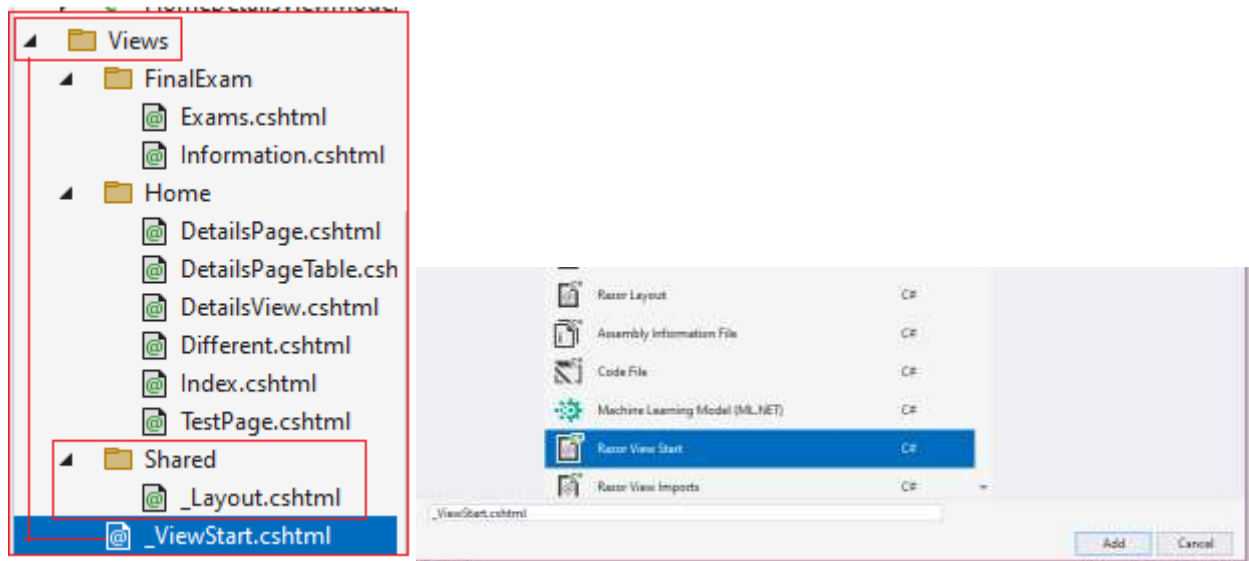
- Consistent look and behaviour for all the views in a web application
- Similar to master page in ASP.NET Web Forms
- File on the file system with **.cshtml** extension
- Default name is **_Layout.cshtml**
- Usually placed in **Views/Shared** folder
- An application can have multiple layout views

За да се избегне повторението на блоков код с извикване на един и същи _Layout във всеки индивидуален View, се използва _ViewStart.cshtml. Кодът във _ViewStart се изпълнява преди всички останали Views.cshtml и има йерархична структура.

_ViewStart in ASP.NET Core MVC

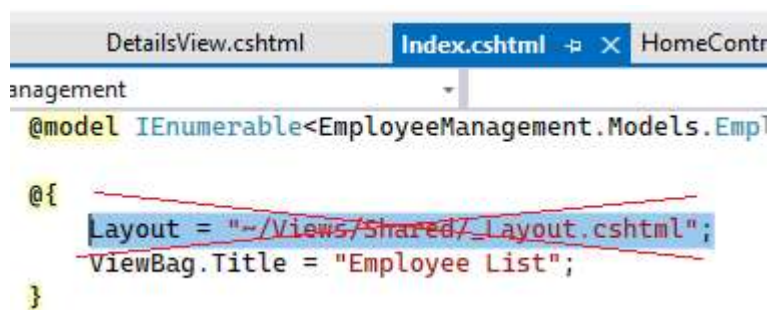
- Code in **ViewStart** is executed before the code in an individual view
- Move the **common code** such as setting the Layout property to ViewStart
- ViewStart reduces **code redundancy** and **improves maintainability**
- ViewStart file is hierarchical





По този начин, управлението става много по-лесно, т.к. трябва да се смени само този 1 ред код, ако искаме да използваме друг _Layout.cshtml, вместо навсякъде. Тази линия повтарящ се в Index, Details вече може да бъде отстранена:

Layout = "~/Views/Shared/_Layout.cshtml";



_ViewStart.cshtml не се извиква никъде, въпреки това, се изпълнява автоматично.

Ще украсим локално таблицата със CSS преди тялото:

```

<style>
    table, th, td {
        border: 1px solid blue;
        background-color: antiquewhite;
    }

```

```

        color: maroon;
        margin: 30px;
        padding: 10px;
    }
</style>

```

```

ml      DetailsView.cshtml  Index.cshtml  HomeController.cs
:Management
@model IEnumerable<EmployeeManagement.Models.Employee>

<style>
    table, th, td {
        border: 1px solid blue;
        background-color: antiquewhite;
        color: maroon;
        margin: 30px;
        padding: 10px;
    }
</style>

@{
    ViewBag.Title = "Employee List";
}

<h1>List of all employees</h1>
<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Family</th>
            <th>Phone</th>
            <th>Email</th>
            <th>Department</th>
        </tr>
    </thead>
    <tbody>

```

local CSS

<html>

<head>

<body>

...

```
nl  _Layout.cshtml  DetailsView.cshtml  Index.cshtml  HomeController.cs
Management
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>@ViewBag.Title</title>
</head>
<body>
  <div>
    @RenderBody()
  </div>
</body>
</html>
```

```
display: block;
background-color: lightblue;
padding: 8px 16px;
text-decoration: none;
color: black;
```

```
background-color: cornflowerblue;
```